Once your environment is configured, this chapter will cover how to write consumer contract tests between a web application and a data provider. If you're also interested in writing contract tests for other consumers such as a mobile and GraphQL client, we'll also cover these consumers, but in chapters 5 and 7. Finally, this chapter will show how you can publish your contract to a central repository, such as Pact Broker, for data providers to access and verify the contract.

By the end of this chapter, you'll not only understand the theory behind contract testing but also have practical experiences in implementing it, making you well-equipped to reap its benefits in your software development projects.

4.1 Guidelines in writing consumer contract tests

Before you start writing consumer contract tests, it's highly important that you take your time to understand recommended practices or guidelines so you don't end up with contract tests that are flaky and brittle. As we mentioned in chapter 2, a test is considered flaky when it produces inconsistent results without introducing any changes to the code. You might run a flaky test several times, and it will pass or fail. On the other hand, a test is considered brittle when it fails due to small code changes or changes from other systems, and it can also be difficult to maintain. Yes, even contract tests can be susceptible to flakiness and brittleness if you're not careful. What you don't want to happen is for your team to start losing confidence in your contract tests. Debugging and fixing flaky and brittle tests can consume valuable development time, slowing the release cycle and potentially causing delays in delivering new features or bug fixes.

Consumer contract tests serve as a critical safety net in your application. They are designed to ensure your services interact correctly and reliably with their data providers. If these tests aren't well-structured or don't follow the recommended practices, they can become unreliable, leading to false positives or negatives, which can also overlook any real problems. This can destroy your team's confidence in the test suite, making it difficult to trust the results. In this section, we'll discuss some